LABORATOIRE CTF 2026

CTF 5 · Ports et serveur de calcul

Scripts Bash et port knocking

Conteneur : menu « ordre » n°58 (ctf_creer.sh), choix CTF5 · énoncé d'origine remis en forme le 07/10/2026, complété le 28/10/2026 après test en conditions réelles, captures d'écran reprises sans retouche.

Avant de commencer. Ce document ne contient aucune adresse réseau réelle : il doit pouvoir servir dans n'importe quelle salle. Deux notations reviennent partout, remplacez-les au fur et à mesure.

— **${TARGET_IP}** : l'adresse IP de la machine cible. Votre professeur vous la communique ; elle est la même pendant tout le lab.

Exemple concret. Si votre professeur vous donne la cible `192.0.2.25`, alors `ping ${TARGET_IP}` se tape `ping 192.0.2.25` — jamais `ping $192.0.2.25`, ni `ping [192.0.2.25]`, ni `ping TARGET_IP`. De même, `ssh utilisateur@${TARGET_IP}` se tape avec le VRAI identifiant que vous avez trouvé, par exemple `ssh alice@192.0.2.25` — jamais `ssh user@...`. L'adresse 192.0.2.25 n'est qu'un exemple : la vôtre sera différente.

« Host key verification failed » : ce n'est pas une panne. Au premier contact avec une machine, SSH affiche son empreinte et vous demande de confirmer : répondez `yes` en toutes lettres — ni `y`, ni `o`, ni `oui`, sinon SSH refuse après trois essais. Si vous aviez déjà accepté cette adresse lors d'un CTF précédent, vous verrez à la place un avertissement `REMOTE HOST IDENTIFICATION HAS CHANGED!` et SSH refusera sans rien demander. C'est normal et voulu : la machine cible a été remplacée par celle du CTF suivant — même adresse, mais identité cryptographique différente. SSH a mémorisé l'ancienne dans `~/.ssh/known_hosts` et vous prévient que son interlocuteur a changé, exactement comme si quelqu'un se faisait passer pour le serveur. Pour repartir, effacez l'ancienne empreinte puis reconnectez-vous et répondez `yes` :

```
ssh-keygen -R ${TARGET_IP}
```

Réflexe professionnel : cette alerte ne se contourne qu'APRÈS avoir compris pourquoi la clé a changé. Ici vous le savez : la machine a été recréée. En entreprise, la même alerte peut signaler une attaque par interposition — on ne la balaie jamais d'un `ssh-keygen -R` sans se poser de question.

1. Création de scripts Bash

© Damien SCONTRINO · professeur BTS SIO, lycée Louis Armand, Nogent-sur-Marne

Introduction à la création de scripts Bash Un script bash est un fichier texte contenant une série de commandes que Bash exécutera. Pour indiquer au système d'exploitation le chemin vers l'interpréteur que nous souhaitons utiliser, nous utilisons le shebang #!/bin/bash au début du fichier. Ainsi, un exemple classique de script "Hello World" ressemblerait à ceci :

```
#!/bin/bash
echo "Hello World!"
```

2. Hello world

— Création et exécution d'un script Bash

— Pour écrire un script bash, créez un nouveau fichier texte à l'aide de l'éditeur de texte de votre choix.

— Exemple en utilisant nano:

— Ouvrez un terminal en cliquant sur l'icône de terminal, puis saisissez la commande suivante :

```
nano helloworld.sh
```

Dans l'éditeur, écrivez votre script, par exemple :

```
#!/bin/bash
echo "Hello World!"
```

Enregistrez le fichier en utilisant la combinaison de touches CTRL + X, puis appuyez sur la touche y (pour confirmer Oui) et enfin appuyez sur la touche Entrée. Vous devez maintenant rendre le script exécutable en saisissant la commande :

```
chmod +x helloworld.sh
```

Enfin, exécutez votre script avec la commande `./helloworld.sh`.

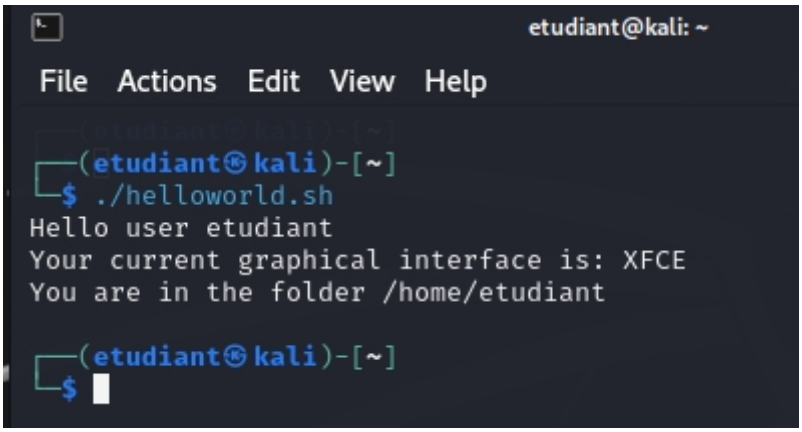
3. Introduction aux variables

Comme dans tous les langages, les variables sont utilisées pour stocker des données. Nous pouvons déclarer (ou définir) une variable et lui attribuer une valeur. Cette valeur peut être un nombre, une chaîne de caractères... Les variables en bash sont généralement déclarées de la manière suivante : clé=valeur.

```
#!/bin/bash
hello="Hello"
echo "$hello user $USER"
echo "Your current graphical interface is: $XDG_CURRENT_DESKTOP"
echo "You are in the folder $PWD"
```

Ici nous avons défini une seule variable (hello) et pourtant nous en avons utilisé 4.

© Damien SCONTRINO · professeur BTS SIO, lycée Louis Armand, Nogent-sur-Marne



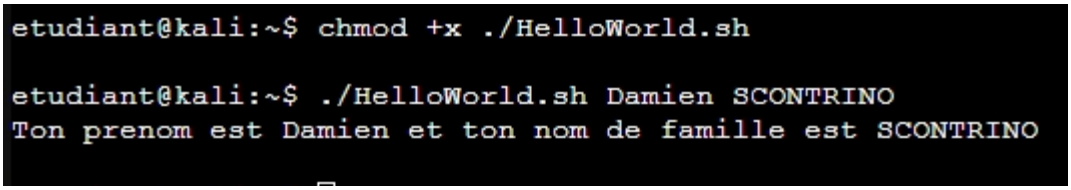
Ces variables sont en fait déjà définies, on les appelle les variables d'environnement. Vous pouvez les lister avec la commande `env`.

4. Passer des arguments au script

Maintenant, utilisons des arguments dans notre programme. Nous allons utiliser les arguments 1 et 2.

```
#!/bin/bash
echo "Ton prenom est $1 et ton nom de famille est $2"
```

Ainsi, lorsque nous utilisons les arguments, ils sont affichés ainsi :



Variables propres aux scripts — voici un récapitulatif de quelques variables intéressantes :

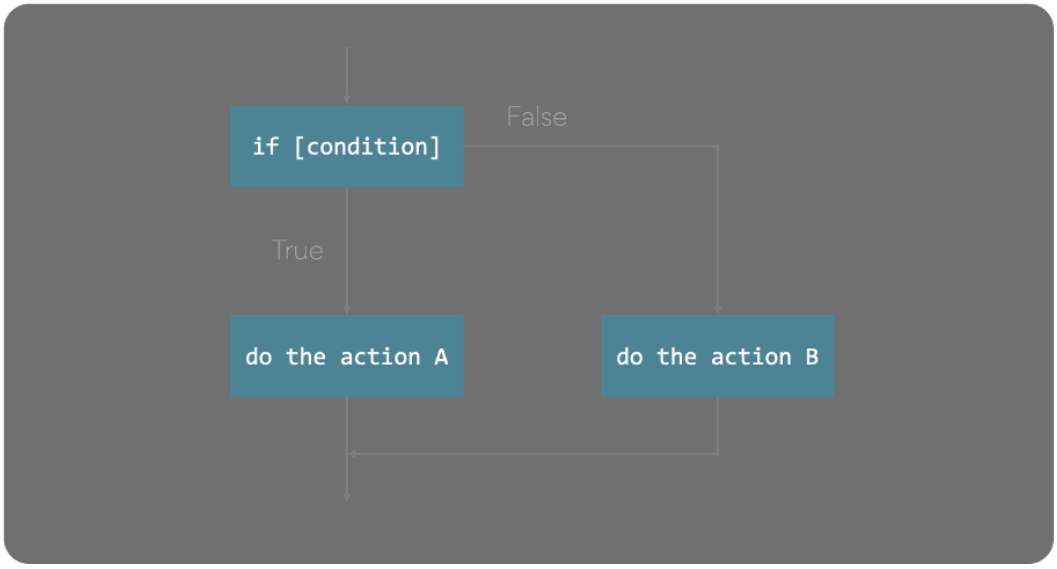
- `$0` contient le nom du script bash.
- `$1` à `$9` contiennent les 9 premiers arguments indiqués à l'appel du script.
- `$#` contient le nombre d'arguments donnés à votre script.
- `$@` est une liste de tous les arguments qui sont passés au script.
- `$RANDOM` est un nombre aléatoire.
- `$?` est le statut de sortie de la dernière commande.
- `$$` est le PID du script actuel.

Entraînez-vous à utiliser ces variables dans votre programme.

5. Introduction aux conditions

Conditions et opérateurs de comparaison Pour contrôler le flux d'exécution d'un programme, les conditions jouent un rôle clé. Voici une représentation visuelle du flux d'exécution :

© Damien SCONTRINO · professeur BTS SIO, lycée Louis Armand, Nogent-sur-Marne



— **Opérateurs de comparaison**

— Voici quelques exemples d'opérateurs de comparaison couramment utilisés :

— `$chaine1 = $chaine2` : vérifie si les deux chaînes sont identiques.

— `$chaine1 != $chaine2` : vérifie si les deux chaînes sont différentes.

— `-z $chaine` : vérifie si la chaîne est vide.

— `-n $chaine` : vérifie si la chaîne est non vide.

— `$num1 -eq $num2` : vérifie si les nombres sont égaux.

— `$num1 -ne $num2` : vérifie si les nombres sont différents.

— `$num1 -lt $num2` : vérifie si num1 est inférieur (<) à num2.

— `$num1 -le $num2` : vérifie si num1 est inférieur ou égal (<=) à num2.

— `$num1 -gt $num2` : vérifie si num1 est supérieur (>) à num2.

— `$num1 -ge $num2` : vérifie si num1 est supérieur ou égal (>=) à num2.

Utilisation des conditions et des comparaisons Ajoutez la condition suivante à votre script :

```
if [[ $1 = "Luke" && $2 = "Skywalker" ]]
then
echo "Bienvenue, $1 $2. Que la force soit avec toi."
else
echo "Tu n'es pas un Jedi."
fi
```

Voici les résultats que vous pouvez obtenir en fonction des arguments passés au script :

© Damien SCONTRINO · professeur BTS SIO, lycée Louis Armand, Nogent-sur-Marne

```
etudiant@kali:~$ ./HelloWorld.sh Damien SCONTRINO
Ton prenom est Damien et ton nom de famille est SCONTRINO
Tu n'es pas un Jedi.

etudiant@kali:~$ ./HelloWorld.sh Luke Skywalker
Ton prenom est Luke et ton nom de famille est Skywalker
Bienvenue, Luke Skywalker. Que la force soit avec toi.
```

6. Boucles : for et while

Boucles : for et while

NB : En programmation, les boucles sont utilisées pour effectuer des tâches répétitives jusqu'à ce qu'une condition spécifique soit remplie. Nous allons voir deux types de boucles couramment utilisés :

— La boucle **for** permet de parcourir une liste de valeurs et d'effectuer une action pour chaque élément de la liste.

— La boucle **while** continue d'exécuter une instruction tant que la condition spécifiée est vraie.

Remarque : ces boucles sont utilisées dans la plupart des langages de programmation.

Utilisation de la boucle for

Voici un exemple en une seule ligne utilisant la boucle for :

```
for num in {10..30..5};do echo $num;done
```

```
etudiant@kali:~$ for num in {10..30..5};do echo $num;done
10
15
20
25
30
```

La boucle **for** ne parcourt pas seulement une plage de nombres : elle peut parcourir n'importe quelle liste de valeurs séparées par des espaces :

```
for mot in un deux trois;do echo $mot;done
```

À chaque tour de boucle, la variable `$mot` vaut l'élément suivant de la liste : le script affiche successivement `un`, puis `deux`, puis `trois`. C'est cette forme de boucle, appliquée à une liste de ports, qui est attendue dans le cas pratique de ce CTF.

À vous de jouer n°1. En une seule ligne, affichez les six ports 12345, 6789, 1569, 6919, 1616 et 5999, un par ligne, à l'aide d'une boucle `for` sur une liste. Vérifiez que les six ports s'affichent dans l'ordre : c'est cette même boucle que vous réutiliserez dans le cas

© Damien SCONTRINO · professeur BTS SIO, lycée Louis Armand, Nogent-sur-Marne

pratique, en remplaçant l'affichage par une requête réseau.

Utilisation de la boucle while

Voici un exemple de la boucle while avec deux conditions séparées par l'opérateur logique "OU". La boucle se répète tant que l'une des deux conditions est vraie. Créez un fichier nommé script_while.sh :

```
nano script_while.sh
```

Ajoutez le contenu suivant dans le fichier script_while.sh :

```
#!/bin/bash
while [ -z $answer ] || [ $answer != 'yes' ]
do
read -p 'Do you want to enter? > ' answer
done
echo "Welcome !"
```


© Damien SCONTRINO · professeur BTS SIO, lycée Louis Armand, Nogent-sur-Marne

```
etudiant@kali:~$ chmod +x script_while.sh

etudiant@kali:~$ ./script_while.sh
Do you want to enter? > no
Do you want to enter? > ok
Do you want to enter? > YES
Do you want to enter? > yes
Welcome !
```

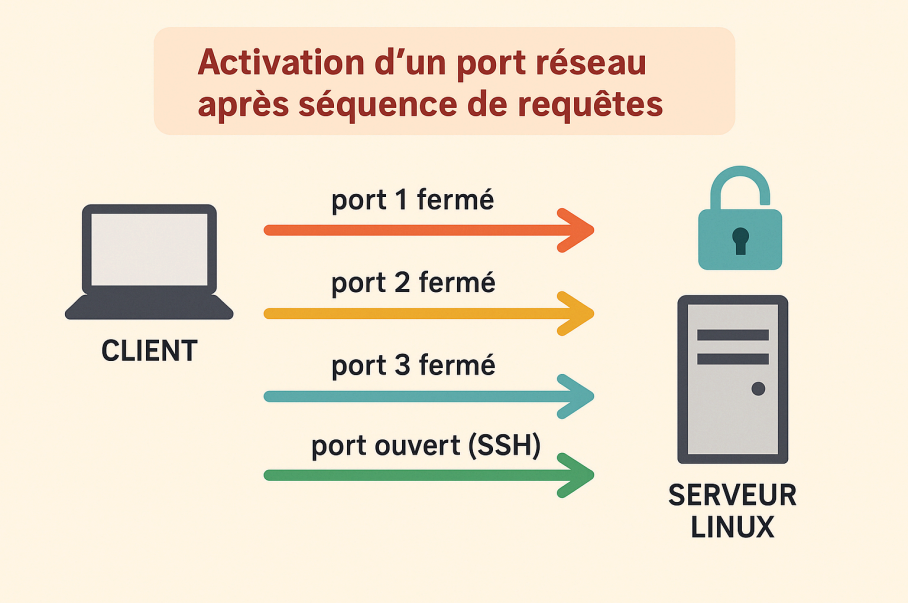
7. Pour aller plus loin

Exercices pratiques pour approfondir vos connaissances en utilisant des boucles, des conditions, des commandes Linux et des variables :

Exercice 1 : Devinez le nombre. Écrivez un script qui demande à l'utilisateur de deviner un nombre choisi aléatoirement au début du programme.

Exercice 2 : Sauvegarde automatique. Créez un script qui permet d'effectuer une sauvegarde automatique d'un répertoire courant. Vous pouvez utiliser la commande `tar` pour cela.

CAS PRATIQUE - Capture the flag



Une machine cible est configurée pour autoriser les connexions SSH si elle reçoit des requêtes sur une série de ports suivant une séquence prédéfinie : c'est une mesure de sécurité destinée à dissimuler un service, qu'on appelle le « port knocking ».

Le port knocking repose sur le principe de « security through obscurity », c'est-à-dire rendre invisible un service sensible (comme SSH) en le dissimulant derrière une séquence d'accès connue uniquement des utilisateurs légitimes.

Cela ne protège pas directement contre une attaque si le service est mal configuré ou si le mot de passe est faible. En revanche, cela réduit fortement la surface d'attaque exposée.

© Damien SCONTRINO · professeur BTS SIO, lycée Louis Armand, Nogent-sur-Marne

L'ANSSI ou les bonnes pratiques en cybersécurité ne recommandent jamais d'utiliser uniquement ce type de protection.

— Un attaquant peut sniffer la séquence si le réseau n'est pas chiffré.

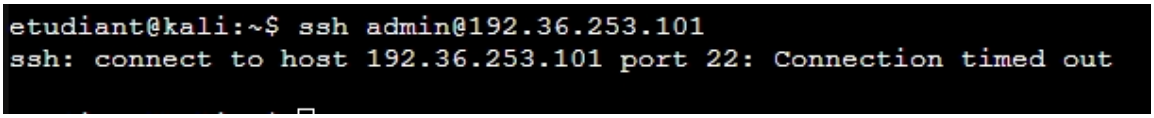
— Si le démon knockd est mal configuré, il peut être manipulé.

— Ce n'est pas une authentification forte, contrairement à des clés SSH, un VPN ou une MFA.

Vous pouvez toujours essayer de vous connecter via SSH avec la commande suivante, mais vous constaterez que le port SSH est fermé :

```
ssh admin@${TARGET_IP}
```

Rappel : ${TARGET_IP} doit être modifié avec l'adresse IP de la machine cible.



Pour vous exercer à utiliser la boucle for, créez un script en utilisant netcat (nc) pour envoyer des requêtes aux ports 12345, 6789, 1569, 6919, 1616 et 5999, dans cet ordre précis. L'argument -z de netcat peut être utilisé pour demander les ports et fermer la connexion, au lieu de laisser la connexion ouverte et bloquer le script.

Indice. La commande `nc` prend d'abord l'adresse de la machine, puis le port : `nc -z 192.0.2.1 22` « toque » au port 22 de la machine 192.0.2.1, puis ferme la connexion grâce à l'argument `-z`. Dans votre script, la boucle parcourt la liste des six ports et envoie une requête `nc -z` sur chacun, à l'adresse `${TARGET_IP}`.

À vous de jouer n°2. Avant d'écrire le script, testez netcat à la main, une seule fois : tapez `nc -z ${TARGET_IP} 12345` (avec votre vraie adresse cible). Rien ne se passera : une séquence incomplète est ignorée, c'est justement le principe du port knocking. Vous venez de « toquer » à une porte ; votre script final fera les six, dans l'ordre.

Vous pouvez soit écrire le script dans un fichier avant de l'exécuter, soit directement dans la console, selon vos préférences.

Utilisez la syntaxe for introduite précédemment dans ce TP.

Rappel : N'oubliez pas de modifier ${TARGET_IP} avec l'adresse IP de la machine cible.

Note : avant d'exécuter le script, autorisez son exécution avec la commande :

```
chmod +x script_name
```

Ensuite, exécutez-le avec la commande :

```
./script_name
```


© Damien SCONTRINO · professeur BTS SIO, lycée Louis Armand, Nogent-sur-Marne

Une fois ce code exécuté, vous pourrez vous connecter en SSH :

```
ssh admin@${TARGET_IP}
```

Rappel : N'oubliez pas de modifier ${TARGET_IP} avec l'adresse IP de la machine cible. Enfin, récupérez le flag et apportez-le à votre professeur afin qu'il valide votre avancée.

> **NB :** le mot de passe du compte admin est volontairement simple. L'objectif de ce CTF est le port knocking, pas le craquage de mot de passe : inutile de lancer une attaque par dictionnaire.

```
cat flag.txt
```

Suite du TP …

Ce cours est la suite de l'initiation au scripting au cours de laquelle vous avez appris les bases telles que les boucles, les conditions, les variables et les arguments. Dans ce cours, nous allons aborder les fonctions, les booléens logiques, les flux, les redirections de flux, ainsi que l'utilisation de quelques commandes qui seront très utiles dans vos futurs projets en bash.

Cas pratique - étape 1

Fonctions en programmation

Les fonctions sont présentes dans la plupart des langages de programmation. Elles permettent d'effectuer une ou plusieurs tâches en prenant des paramètres en entrée. Vous pouvez créer autant de fonctions que vous le souhaitez. Voici quelques points importants à connaître :

— 1. Chaque fonction d'un même programme doit avoir un nom différent pour éviter les conflits avec les fonctions globales.

— 2. Les fonctions peuvent utiliser les variables globales initialisées dans votre script, à l'intérieur ou à l'extérieur de la fonction. Il est également possible de déclarer des variables locales spécifiques à une fonction.

— 3. Il est recommandé de spécifier une valeur de retour pour les fonctions, par exemple 0 pour indiquer que tout s'est bien passé.

Une autre bonne pratique consiste à créer des fonctions pour traiter des éléments spécifiques et récurrents dans votre script.

Déclaration d'une fonction

Il existe deux manières de déclarer une fonction en bash :

```
func () {
instructions
}
# Appel de la fonction
```


© Damien SCONTRINO · professeur BTS SIO, lycée Louis Armand, Nogent-sur-Marne

```
func
```

OU

```
function func2 {
instructions
}
# Appel de la fonction
func2
```

Exemple

Voici un exemple d'utilisation d'une fonction qui réécrit la commande `ls` :

© Damien SCONTRINO · professeur BTS SIO, lycée Louis Armand, Nogent-sur-Marne

```
etudiant@debian12:~$ ./funct.sh
total 100K
drwx------ 17 etudiant etudiant 4,0K  7 juin  19:09 .
drwxr-xr-x  3 root     root     4,0K  3 déc.   2024 ..
-rw-------  1 etudiant etudiant 2,6K 20 mai   19:02 .bash_history
-rw-r--r--  1 etudiant etudiant  220  3 déc.   2024 .bash_logout
-rw-r--r--  1 etudiant etudiant 3,5K  3 déc.   2024 .bashrc
drwxr-xr-x  2 etudiant etudiant 4,0K  3 déc.   2024 Bureau
drwx------ 11 etudiant etudiant 4,0K 13 mai   20:18 .cache
drwx------ 11 etudiant etudiant 4,0K 20 mai   13:47 .config
drwxr-xr-x  2 etudiant etudiant 4,0K  3 déc.   2024 Documents
-rw-r--r--  1 etudiant etudiant 5,2K  3 déc.   2024 .face
lrwxrwxrwx  1 etudiant etudiant    5  3 déc.   2024 .face.icon -> .face
-rwxr-xr-x  1 etudiant etudiant   47  7 juin  19:09 funct.sh
drwx------  2 etudiant etudiant 4,0K 13 mai   20:12 .gnupg
drwxr-xr-x  2 etudiant etudiant 4,0K  3 déc.   2024 Images
drwx------  4 etudiant etudiant 4,0K  3 déc.   2024 .local
drwxr-xr-x  2 etudiant etudiant 4,0K  3 déc.   2024 Modèles
drwx------  4 etudiant etudiant 4,0K 13 mai   20:18 .mozilla
drwxr-xr-x  2 etudiant etudiant 4,0K  3 déc.   2024 Musique
-rw-r--r--  1 etudiant etudiant  807  3 déc.   2024 .profile
drwxr-xr-x  2 etudiant etudiant 4,0K  3 déc.   2024 Public
drwx------  2 etudiant etudiant 4,0K  3 déc.   2024 .ssh
-rw-r--r--  1 etudiant etudiant    0 20 mai   13:33 .sudo_as_admin_successful
drwxr-xr-x  2 etudiant etudiant 4,0K 20 mai   13:51 Téléchargements
drwxr-xr-x  2 etudiant etudiant 4,0K 20 mai   13:37 veyon-install
drwxr-xr-x  2 etudiant etudiant 4,0K  3 déc.   2024 Vidéos
-rw-r--r--  1 etudiant etudiant  180 20 mai   13:36 .wget-hsts
etudiant@debian12:~$
```

Pour aller plus loin, vous pouvez ajouter une fonction qui sauvegarde les résultats ou les journaux dans votre script.

À vous de jouer n°3. Écrivez un script qui déclare une fonction `saluer()` affichant `Bonjour !`, puis appelez-la trois fois : à l'exécution, le message doit apparaître trois fois. Essayez ensuite d'appeler la fonction AVANT sa déclaration dans le script : que se passe-t-il, et pourquoi ?

Opérateurs logiques

Nous allons maintenant explorer les opérateurs logiques booléens, les flux et les processus. Les opérateurs logiques booléens sont très utilisés en bash. Ils incluent les opérateurs `&&` (ET) et `||` (OU). Ils permettent d'exécuter des commandes en fonction de conditions. Par exemple, tapez ceci dans votre terminal :

```
grep $USER /etc/passwd && echo "$USER found!"
```

puis

```
grep notuser /etc/passwd && echo "notuser found!"
```

La deuxième commande n'est pas exécutée si la première ne retourne pas "vrai".

Entrées standards

Il y a 3 entrées standards dans les processus : l'entrée standard (stdin), la sortie standard (stdout) et la sortie d'erreur (stderr). Ces flux peuvent être redirigés de plusieurs manières, et les commandes peuvent être enchaînées en utilisant le pipe `|`.

© Damien SCONTRINO · professeur BTS SIO, lycée Louis Armand, Nogent-sur-Marne

Par exemple, la commande suivante vous affichera toutes les références dans le fichier index.html :

```
grep href /var/www/html/index.html
```

Vous pouvez également utiliser le chevron > ou le double chevron >> pour rediriger le flux de sortie. Tapez :

```
echo "Hello Mr Y" > test
```

puis :

```
echo "Hello Mr Y" >> test
```

Que s'est-il passé ?

Il est possible de rediriger une entrée/sortie standard vers une autre (par exemple, stdout vers stderr). La syntaxe pour cela est : `1>&2`.

Créez le script suivant :

```
#!/bin/bash
echo "Redirect this STDOUT to STDERR" 1>&2
```

Et faites les tests suivants :

```
./yourscript.sh > stdout.txt
./yourscript.sh 2> stderr.txt
```

Que se passe-t-il ?

> **NB :** pour rediriger stderr vers stdout, la syntaxe serait : `2>&1`.

Ces concepts sont importants à comprendre. Par exemple, lors d'une exploitation web, il est souvent nécessaire de rediriger le flux d'erreur vers la sortie standard pour obtenir des informations sur cette exploitation.

Deux notations encore vont servir dans le dernier script. La substitution de commande `$(commande)` exécute la commande et remplace le tout par sa sortie : `echo "Nous sommes le $(date)"`. Le calcul arithmétique `$((expression))` évalue l'expression entre les doubles parenthèses : `$((7*6))` vaut 42.

> **À vous de jouer n°4.** Affichez le résultat de `$((12*3))` dans un `echo` ; puis stockez-le dans une variable `resultat` et affichez « Résultat : 36 » ; enfin, testez `rep=$(whoami)` puis `echo $rep` : votre nom d'utilisateur doit s'afficher. Ces deux notations sont celles du script final.

Cas pratique - étape 2

© Damien SCONTRINO · professeur BTS SIO, lycée Louis Armand, Nogent-sur-Marne

Attention : Commandes groupées en bash. Les commandes groupées sont un concept important en bash. Elles permettent d'exécuter une série de commandes dans le shell actuel, dans le même environnement. On peut les écrire selon la syntaxe suivante : { cmd1; cmd2; cmd3 }. Il existe également une autre syntaxe pour exécuter les commandes dans un sous-shell : (cmd1; cmd2; cmd3), ou utiliser des backticks : cmd1. Attention à ce comportement, notamment pour les variables ! **Exercice pratique pour aller plus loin.** Pour aller plus loin dans votre maîtrise de bash, vous pouvez modifier votre .bashrc afin d'automatiser certaines tâches et optimiser certaines fonctions existantes en utilisant des alias. N'hésitez pas à expérimenter les redirections de flux et les commandes groupées. Automatisez vos tâches rébarbatives !

Récupération du Flag

Voici un exemple de script Bash pour récupérer le "flag" :

```
#!/bin/bash
# Adresse et port du serveur de destination
host=$1
port="54321"
# Répondre merci au serveur et récupérer le calcul complet
calc="$(echo 'Merci!' | nc -q2 $host $port| tail -n 1)"
# Récupérer l'opérateur du calcul
op="$(echo 'op' | nc -q2 $host $port| tail -n 1)"
n1="$(echo 'n1' | nc -q2 $host $port| tail -n 1)"
n2="$(echo 'n2' | nc -q2 $host $port| tail -n 1)"
#TO-DO Fonction exécutée lorsque l'opérateur de calcul est "+"
function addition()
{
}
#TO-DO Fonction exécutée lorsque l'opérateur de calcul est "-"
function soustraction()
{
}
#TO-DO Fonction exécutée lorsque l'opérateur de calcul est "*"
function multiplication()
{
}
function exemple()
{
echo "Le résultat est : $(($n1 * $n2))"
flag=$(echo $(($n1 * $n2)) | nc -q2 $host $port| tail -n 1)
echo $flag
exit 0
}
if [[ $op == "+" ]] ; then
# Exécution de la fonction "addition"
addition
elif [[ $op == "-" ]] ; then
```


© Damien SCONTRINO · professeur BTS SIO, lycée Louis Armand, Nogent-sur-Marne

```
# Exécution de la fonction "soustraction"
soustraction
elif [[ $op == "*" ]] ; then
# Exécution de la fonction "multiplication"
multiplication
else
echo "Une erreur s'est produite"
echo $calc
echo $op
exit 1
fi
```

Aidez-vous de la fonction `exemple()` pour compléter les trois fonctions contenant le commentaire `#TO-DO` : elles sont identiques à `exemple()` mot pour mot, et la seule différence est l'opérateur du calcul (+, - ou *). Rappel : `$(commande)` exécute la commande et récupère sa sortie ; `$((calcul))` effectue le calcul arithmétique écrit entre les doubles parenthèses. Une fois vos trois fonctions complétées, supprimez la fonction `exemple()` du script : elle ne servait que de modèle. Ensuite, exécutez le script en lui donnant l'adresse IP du serveur comme argument :

```
./script.sh ${IP_SERVEUR}
```

Note : ${IP_SERVEUR} doit être remplacé par l'adresse IP de votre cible. Ce script permettra de récupérer un calcul, d'exécuter la fonction appropriée en fonction de l'opération demandée, puis de renvoyer la réponse au serveur. Le serveur répondra en envoyant le "flag", nécessaire pour valider le TD. Si vous rencontrez des difficultés, un exemple de solution est présenté ci-dessous.

Exemple de solution

Voici un exemple de solution d'un programme utilisant des fonctions pour effectuer des opérations mathématiques.

```
# Fonction exécutée lorsque l'opérateur de calcul est "+".
function addition()
{
echo "Le résultat est : $(($n1 + $n2))"
flag=$(echo $(($n1 + $n2)) | nc -q2 $host $port| tail -n 1)
echo $flag
exit 0
}
# Fonction exécutée lorsque l'opérateur de calcul est "-".
function soustraction()
{
echo "Le résultat est : $(($n1 - $n2))"
flag=$(echo $(($n1 - $n2)) | nc -q2 $host $port| tail -n 1)
echo $flag
exit 0
}
```


© Damien SCONTRINO · professeur BTS SIO, lycée Louis Armand, Nogent-sur-Marne

```
# Fonction exécutée lorsque l'opérateur de calcul est "*".
function multiplication()
{
echo "Le résultat est : $(($n1 * $n2))"
flag=$(echo $(($n1 * $n2)) | nc -q2 $host $port| tail -n 1)
echo $flag
exit 0
}
```


© Damien SCONTRINO · professeur BTS SIO, lycée Louis Armand, Nogent-sur-Marne